

Light Hash algorithm v5

Project

Design a hash module which generates a 64-bit digest for each 64-bit input data block, performing the following operations:

```
for (r = 0; r < 36; r++) {  
    H = SA(m);  
    H = Θ(H);  
    H = ρ(H);  
}  
d = FPX(H);
```

Where:

- r is the number of rounds;
- m is the message;
- d is the digest of the message;
- SA is the State Array function, which transforms the 64-bit input data block m in an array of 8 elements, $m[i]$, each of 1 byte. The SA function works as follows:

```
for (i = 0; i < 8; i++)  
    H[i] = m[i] ⊕ IV[i];
```

	IV[0]	IV[1]	IV[2]	IV[3]	IV[4]	IV[5]	IV[6]	IV[7]
Init. value	0x34	0x55	0x0F	0x14	0xCC	0xAA	0xF0	0xE3

- Θ is the Θ function which permutes the input array as it follows:

```
for (i = 0; i < 8; i++)  
    H[i] = H[7 - i];
```

- ρ is the ρ function which works as it follows:

```
for (i = 0; i < 8; i++)  
    H[i] = (H[i] + 0x85) mod 0xFD;
```

- FPX Is the Final Permutation and Xoring function, which works as it follows:

```
for (i = 0; i < 8; i++)  
    d[i] = (H[7 - i]) ⊕ IV[i];
```

Additional design specifications

- The module shall have an asynchronous active-low reset port.
- The module interface should include appropriate flags to indicate the beginning and/or the end of a message, and, accordingly, the module should integrate appropriate logic for counting the bytes of the message.
- The module interface shall include an input flag to be driven as it follows: 1'b1, when input message byte on the corresponding input port is valid and stable (i.e. it can be used by the internal module logic), 1'b0, otherwise.

The module interface shall include an output flag to be driven as it follows: 1'b1, when output digest on the corresponding output port is ready and stable (i.e., external modules can read and use it), 1'b0, otherwise.